# Data Type in C & C++

## Primitive Data Types (Basic)

- Void (no data)
- bool (1 byte, requires <stdbool.h>)
- char (1 byte)
- short (2 bytes)
- int (4 bytes)
- long (4 or 8 bytes)
- long long (8 bytes)
- float (4 bytes)
- double (8 bytes)
- long double (10, 12, or 16 bytes)

## Non-primitive Data Types (Derived)

- Arrays (arr[])
- pointers (*ptr)
- Function (void myfun() {})
- Reference (c++)

## User-Defined Data Types

- Structure (struct person { char name[20]; int age ; };)
- Union (union Data { int x; float y; };)
- Enum (enum colors { RED, GREEN, BLUE };)
- Typedef (typedef unsigned int uint;)
- class (c++)

C → Low-level, machine-dependent, no bool. manual memory handling.

C++ → Extends C with OOP, adds bool, still machine-dependent sizes.

Java → High-level, machine-independent, strict data types, Unicode chars, automatic memory management.

# Data Types in Java

- Primitive Data Types
  - Non-Numeric Type
    - Boolean (1 byte)
    - char (2 bytes)
  - Numeric Type
    - Integer
      - byte (1 byte)
      - short (2 bytes)
      - int (4 bytes)
      - long (8 bytes)
    - Floating Point
      - float (4 bytes)
      - double (8 bytes)
- Non-Primitive Data Types
  - String
  - Array
  - etc. Interface
  - Object
  - Class

Java is statically typed programming language which means variables types are known at the compiler time.

- In Java, compiler knows exactly what types each variable holds and enforces correct usage during compilation. For example. "int x = "Gfg";" gives compiler error in Java as we are trying to store string in an Integer type.

- Data types in Java are of different sizes and values that can be stored in a variable that is made as per convenience and circumstances to handle different scenarios or data requirement.

# Data Type in Javascript

## primitive Data Types

- Undefined (let x;)
- Null (let y = null;)
- Number ((int, float) let num = 10;)
- String (let name = "Prachi";)
- Boolean (let flag = true;)
- BigInt (let big = 123n;)
- Symbol (let sym = Symbol("id");)

## Non-primitive Data Types

- Object (let obj = {name : "Prachi", age: 22};)
- Array (let arr = [1,2,3];)
- function (function greet() {return "Hello";})

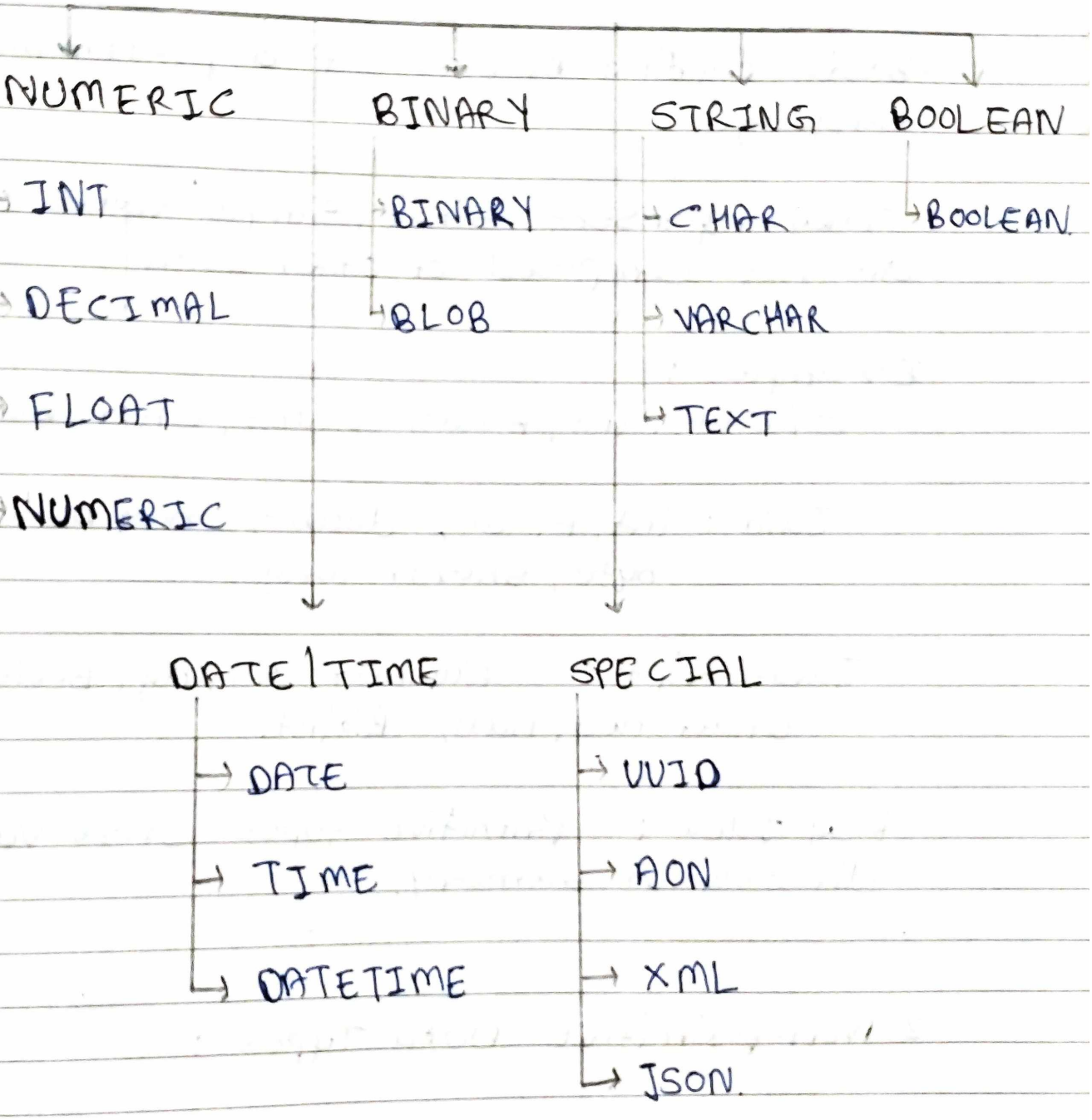
## User-Defined Data Types

- class (class car {constructor(name) {this.name = name;}})
- custom objects (let person = {name : "John", age: 30};)

# SQL Commands

- DDL
  - CREATE
  - ALTER
  - DROP
  - RENAME
  - TRUNCATE
  - COMMENT
- DML
  - SELECT
  - INSERT
  - UPDATE
  - DELETE
  - MERGE
  - CALL
  - EXPLAIN PLAN
  - LOCK TABLE
- DCL
  - GRANT
  - REVOKE
- TCL
  - COMMIT
  - ROLLBACK
  - SAVEPOINT
  - SET TRANSACTION

# SQL DATA Types



1. Primitive Data Types :

- Basic building blocks of a programming language.

- They represent single, simple values and are not composed of other data.

Examples :
C / C++ : int, float, double, char, bool.

Java : int, float, double, char, boolean, byte, short, long.

JavaScript : Number, String, Boolean, undefined, null, Bigint.

- Key Idea : Primitive types store values directly in memory.

2. Non-Primitive Data Types :

- More complex data types built using primitive types.

- They can hold multiple values or structured data.

Examples :

c / c++ : Arrays, structs, classes, pointers.

Java : Arrays, classes, objects, interfaces, strings.

JavaScript : objects, Arrays, functions.

- Key Idea : Non-primitive types often store references (addresses) to data rather than the raw value.

# Data Types KeyNotes :—

In Javascript, we cannot use data types to declare variable. we use let, var, const keyword.

Ex. let num = 10;

In Javascript, Variable datatypes can even change later.

Ex. let x = 5; // number
x = "Hello"; // String.

In c, we must declare the datatype (eg. int, float, char etc.,)

Ex. int num = 10;

In c, Variable datatypes cannot change later.

Ex. int x = 5;
x = "Hi"; // Error.

In c++, The keyword auto tells the compiler to automatically detect (infer) the variable's data type from the value you assign to it.

In c++, Similar as c, we can declare variable with datatype or using auto keyword.

Ex. int num = 10;
auto age = 25;

In C++, variable datatypes cannot change later.

Ex. auto age = 25;
age = "twenty-five"; // Error.

In Java, we must declare the type of each variable.

Ex. int num = 100;

Java is strictly typed, so data type cannot change.

Ex. char name = 'P'; // char
name = "prajakta"; // Error.

In Java, variables must belong to a scope (inside class/ method).

Variable is a container of store data values like number, alphabet, symbol etc.,

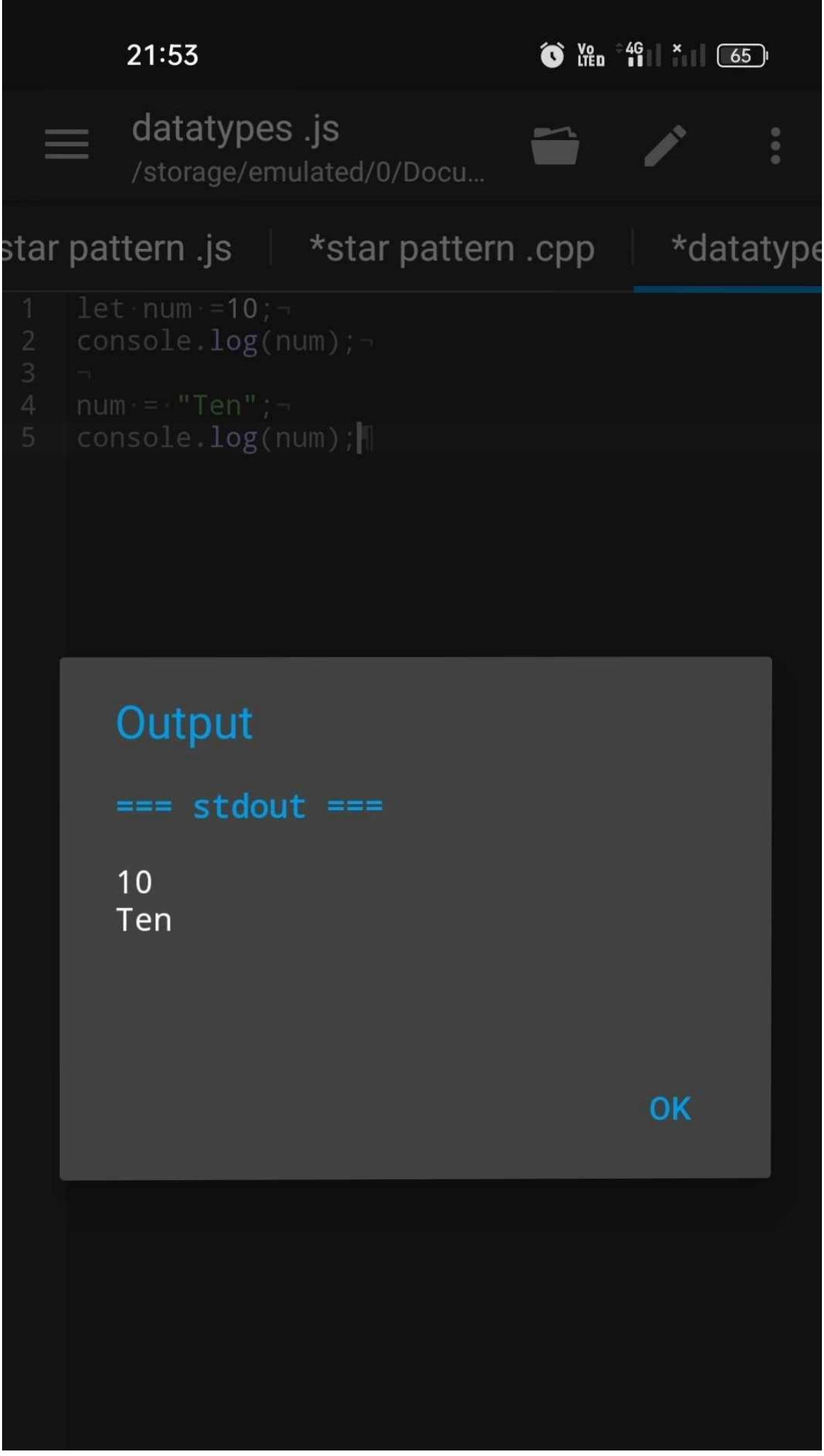
Page No.
Date

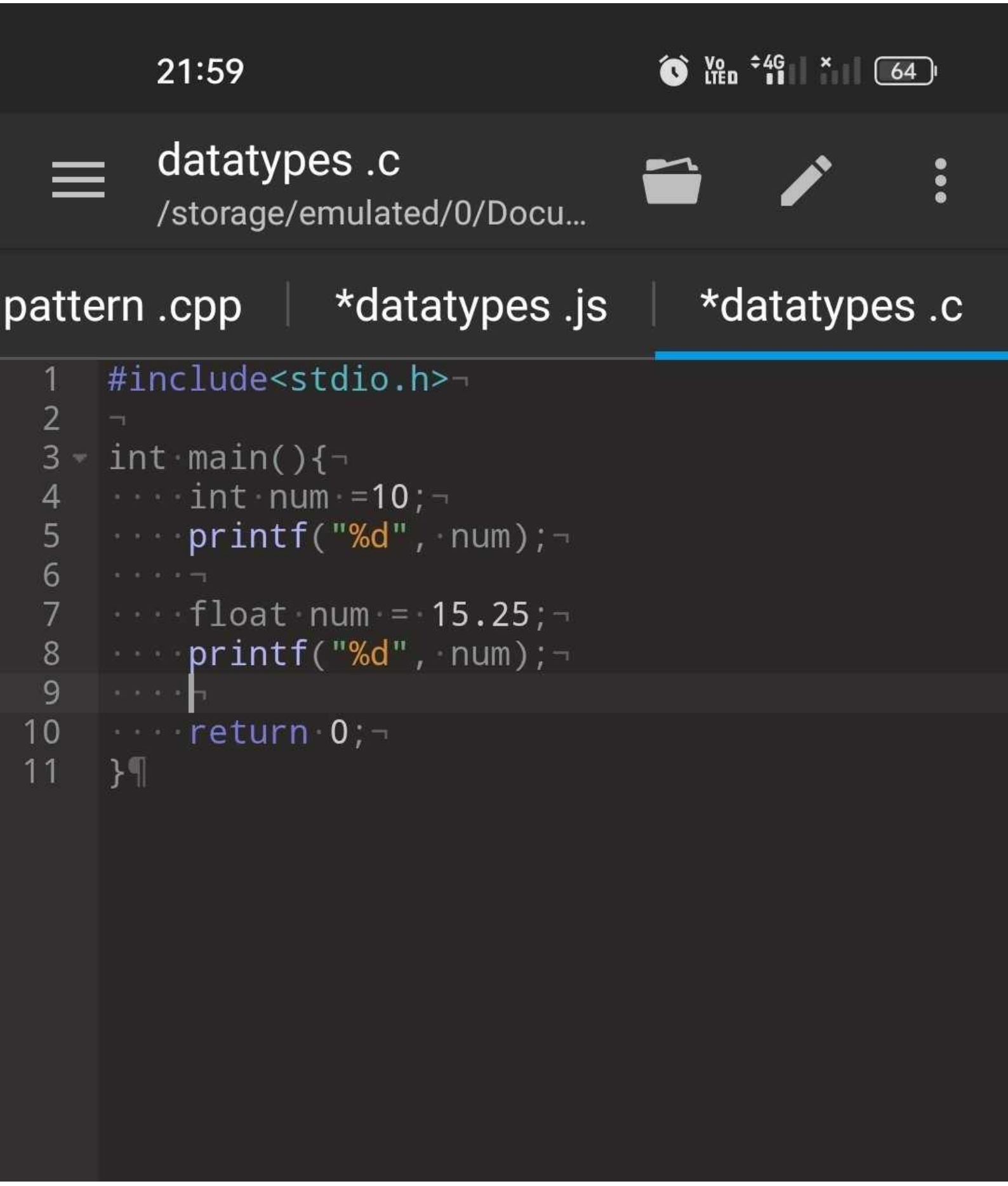
## Comparison summary

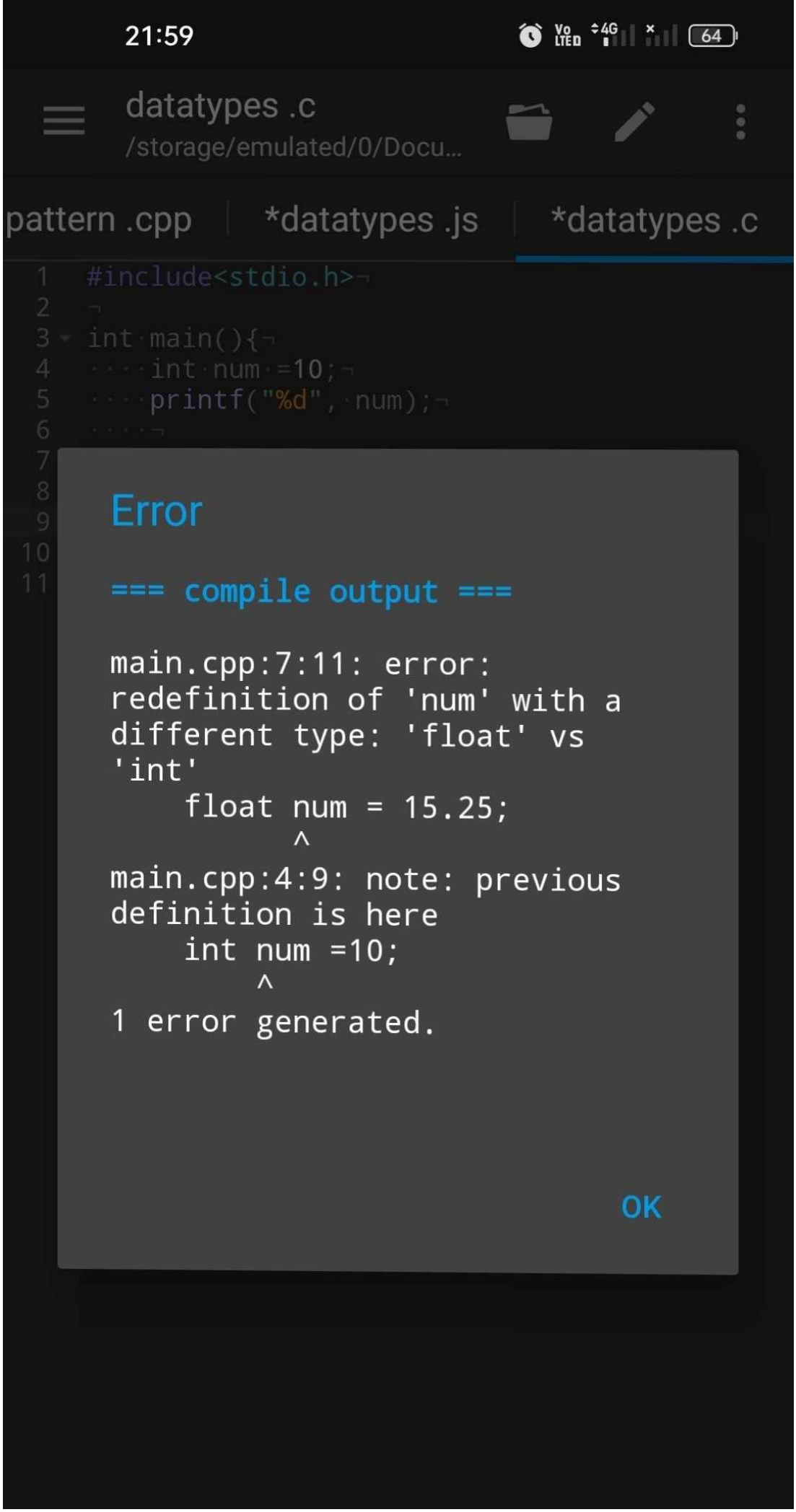
| Language | Type System | How to declare | Can change Type Later? |
|---|---|---|---|
| Javascript | Dynamic | let, var, const | Yes |
| C | Static | int, float char etc, | No |
| C++ | static + Type inference (auto) | int, float, auto etc, | No |
| Java | Strictly static | int, double string etc. | No |

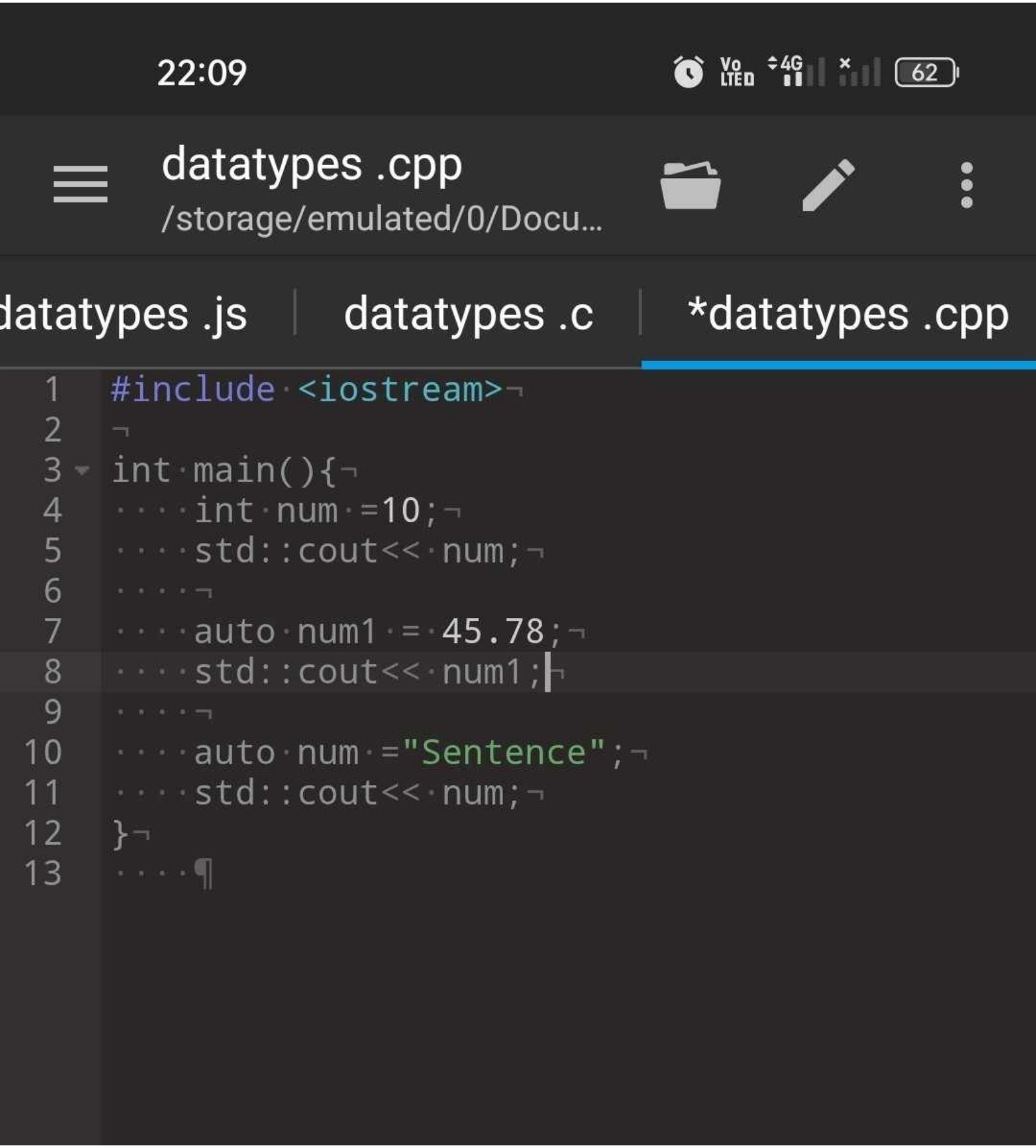
## Key Takeaway :

| Concept | Javascript | C/C++/Java. |
|---|---|---|
| Type checking | At runtime | At compile-time |
| Variable keywords | Var, let, const | Type name (int, float) |
| Type flexibility | Can change type anytime | Fixed once declared. |
| memory management | Automatic | Manual / Partially automatic. |


21:53
datatypes .js
/storage/emulated/0/Docu...
star pattern .js
*star pattern .cpp
*datatype
let num =10;
console.log(num);
num = "Ten";
console.log(num);
Output
=== stdout ===
10
Ten
OK


21:59
datatypes .c
/storage/emulated/0/Docu...
pattern .cpp
*datatypes .js
*datatypes .c
#include<stdio.h>
int main(){
int num =10;
printf("%d", num);
float num = 15.25;
printf("%d", num);
return 0;
}


21:59
datatypes .c
/storage/emulated/0/Docu...
pattern .cpp
*datatypes .js
*datatypes .c
#include<stdio.h>
int main(){
int num =10;
printf("%d", num);
Error
=== compile output ===
main.cpp:7:11: error:
redefinition of 'num' with a
different type: 'float' vs
'int'
float num = 15.25;
^
main.cpp:4:9: note: previous
definition is here
int num =10;
^
1 error generated.
OK


22:09
datatypes .cpp
/storage/emulated/0/Docu...
datatypes .js
datatypes .c
*datatypes .cpp
#include <iostream>
int main(){
int num =10;
std::cout<< num;
auto num1 = 45.78;
std::cout<< num1;
auto num ="Sentence";
std::cout<< num;
}

# Error

```
=== compile output ===

main.cpp:10:10: error:
redefinition of 'num'
    auto num ="Sentence";
         ^
main.cpp:4:9: note: previous
definition is here
    int num =10;
        ^
1 error generated.
```

OK

```
1  // Online Java Compiler
2  // Use this editor to write, compile
       and run your Java code online
3
4  class Main {
5      public static void main(String[]
           args) {
6          int num =100;
7          System.out.println(num);
8
9          float num = 568.96;
10         System.out.println(num);
11         System.out.println("Try
               programiz.pro");
12     }
13 }
```

```
ERROR!
Main.java:9: error: variable num is
    already defined in method main
    (String[])
ERROR!
        float num = 568.96;
              ^
Main.java:9: error: incompatible types:
    possible lossy conversion from double
    to float
        float num = 568.96;
                    ^
2 errors

=== Code Exited With Errors ===
```

# Type Casting Key Notes

Type casting means converting one data type into another.

Ex. `int x = 10;`
`float y = (float) x;`

Type casting in Javascript.
Dynamic Typing → data types are decided at runtime.

Implicit (Automatic) :—
Javascript automatically converts between types when needed.

Ex:

```
let x = "5" + 2; // "52" number 2 convert string
let y = "5" - 2; // 3 string "5" convert to num
```

Explicit (manual)
We can convert manually using built-in functions:

Ex;

```
let floatVal = parseFloat("3.14");
```

Javascript automatically handles type conversion → but this can sometimes cause bugs due to "type coercion".

Type Casting in C Language

Static Typing → type must be known at compile-time.

Implicit Casting :—

Done automatically when converting between compatible types:

Ex:

```
int a = 5;
float b = a; // int → float
```

Explicit Casting :—

We can force conversion using parentheses:

Ex:

```
int a = 5, b = 2;
float result = (float) a/b; //2.5
```

without (float), result would be integer division.

Type Casting in C++ Language

Same as C, but C++ adds safe casting style.

Ex:

```
int x = 10;
double y = static_cast<double>(x);
```

There are multiple casting operators in C++:

| Cast | Purpose |
|---|---|
| 1) static_cast | normal conversions (int → float etc) |
| 2) dynamic_cast | for polymorphic classes. |
| 3) const_cast | add/remove constness |
| 4) reinterpret_cast | low-level pointer conversions. |

Type Casting in Java
Strictly typed — all conversions are checked at compile time.

Implicit (widening conversion) :—
Automatically converts smaller → larger data type.

```
Ex.  int a = 10;
     double b = a;
```

Explicit (Narrowing conversion) :—
must be done manually using parentheses:

```
Ex.  double x = 9.78;
     int y = (int) x; //(loses decimals)
```

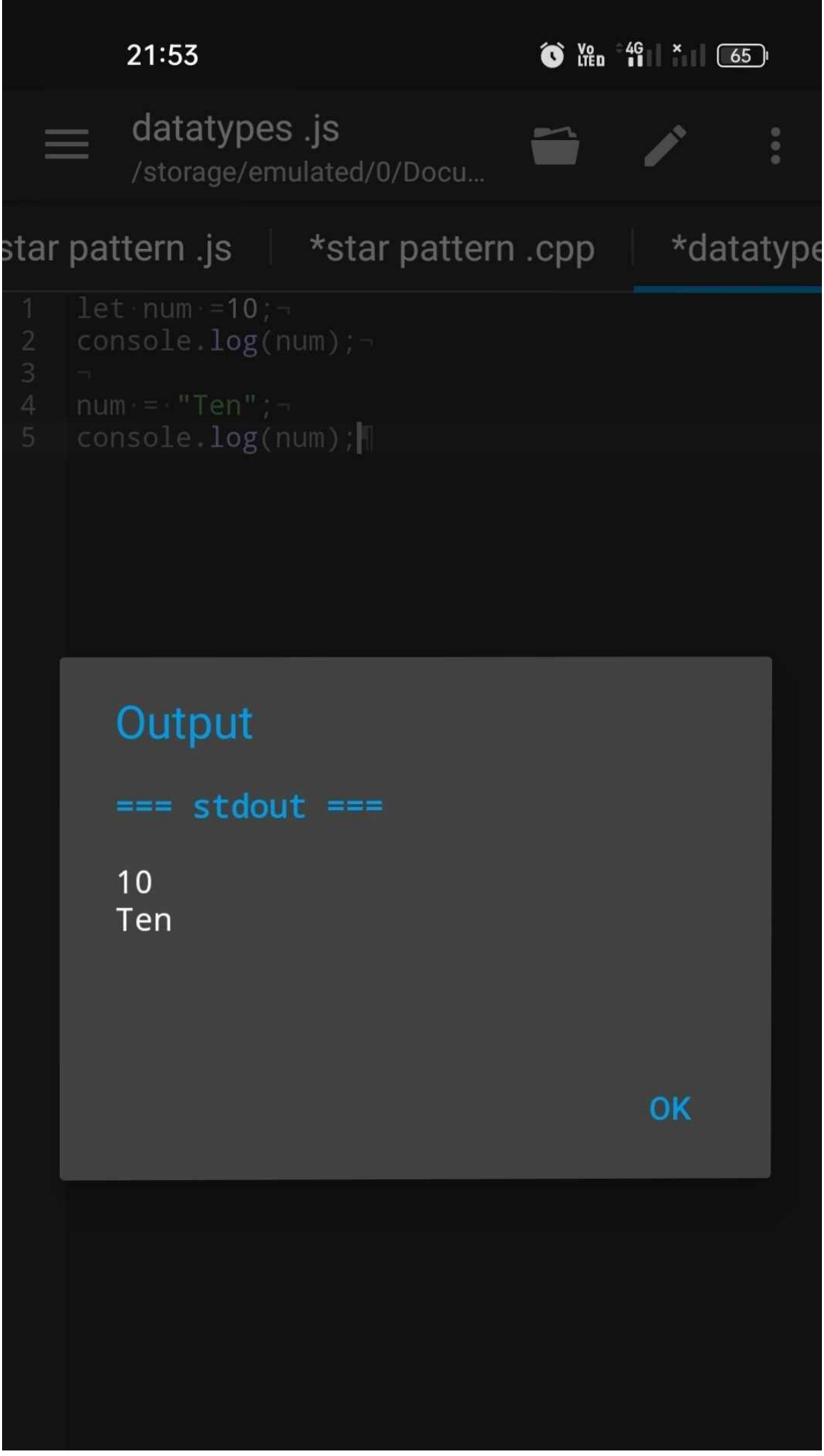
widening (safe) happens automatically,
narrowing (unsafe) needs explicit casting.

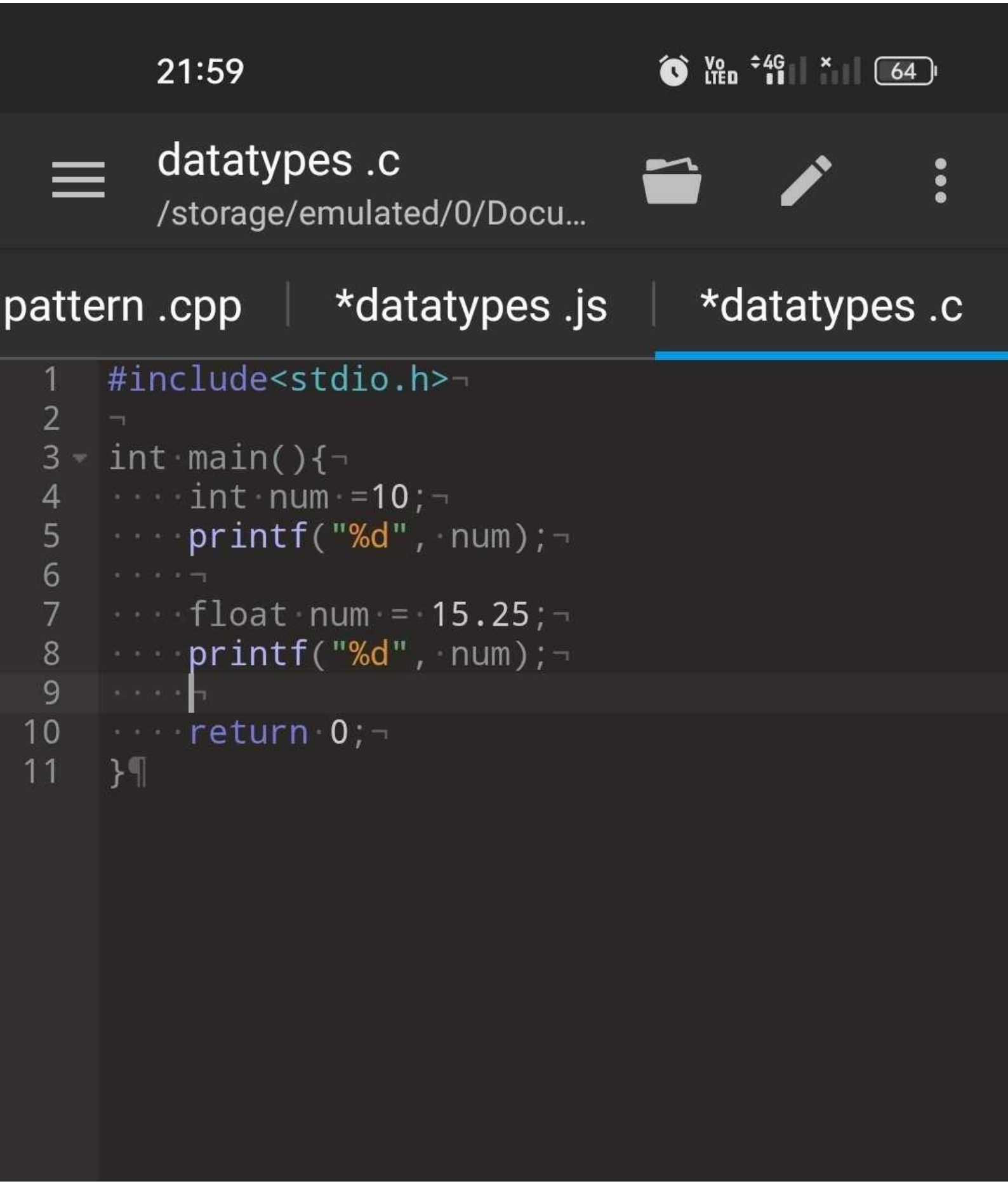
All four Language support type-casting — but in C, C++, and Java, it's strict and compile-time, while in Javascript, it's flexible and runtime-based

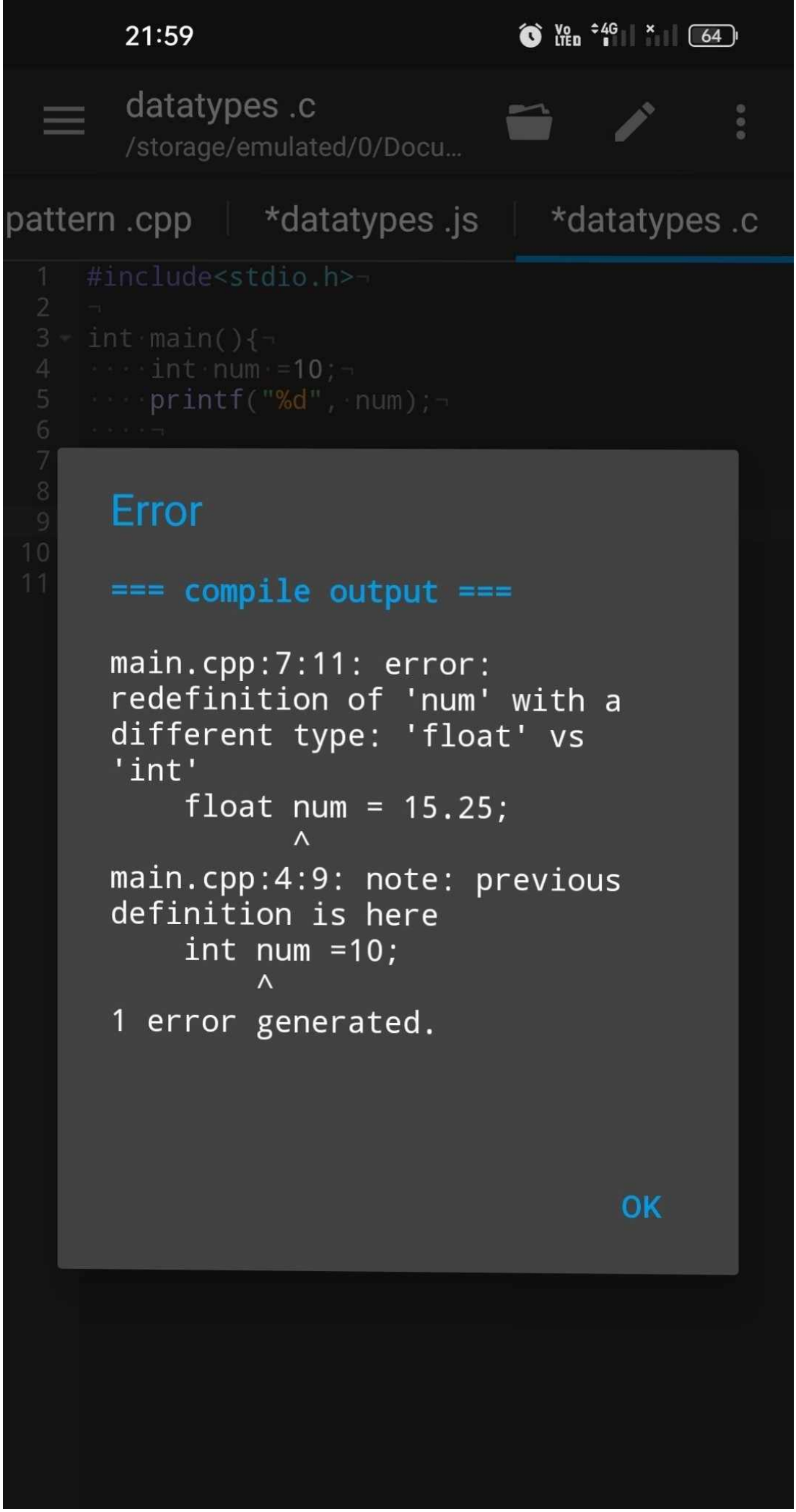
Comparison Table

| feature | Javascript | C | C++ | Java |
|---|---|---|---|---|
| Type System | Dynamic | Static | Static | Strongly Static |
| Implicit Casting | Yes (auto coercion) | Yes | Yes | Yes (widening) |
| Explicit Casting | Using Number(), String(), etc. | (type) | (type) or static_cast<> | (type) |
| when decided | Runtime | Compile-time | Compile-time | Compile-time |
| Example | Number("123") | (float) a | static_cast<float>(a) | (int) x |
| Risk of data loss | Possible (silent coersion) | Possible | Possible | Compiler warns if unsafe. |

| Language | Type Casting style | Like saying |
|---|---|---|
| C/C++/Java | "Tell compiler exactly what type it is" | "change the container type before using". |
| Javascript | "Interpreter guesses or converts automatically" | "Let me handle it dynamically". |


21:53
datatypes .js
/storage/emulated/0/Docu...
star pattern .js
*star pattern .cpp
*datatype
let num =10;
console.log(num);
num = "Ten";
console.log(num);
Output
=== stdout ===
10
Ten
OK


21:59
datatypes .c
/storage/emulated/0/Docu...
pattern .cpp
*datatypes .js
*datatypes .c
#include<stdio.h>
int main(){
int num =10;
printf("%d", num);
float num = 15.25;
printf("%d", num);
return 0;
}

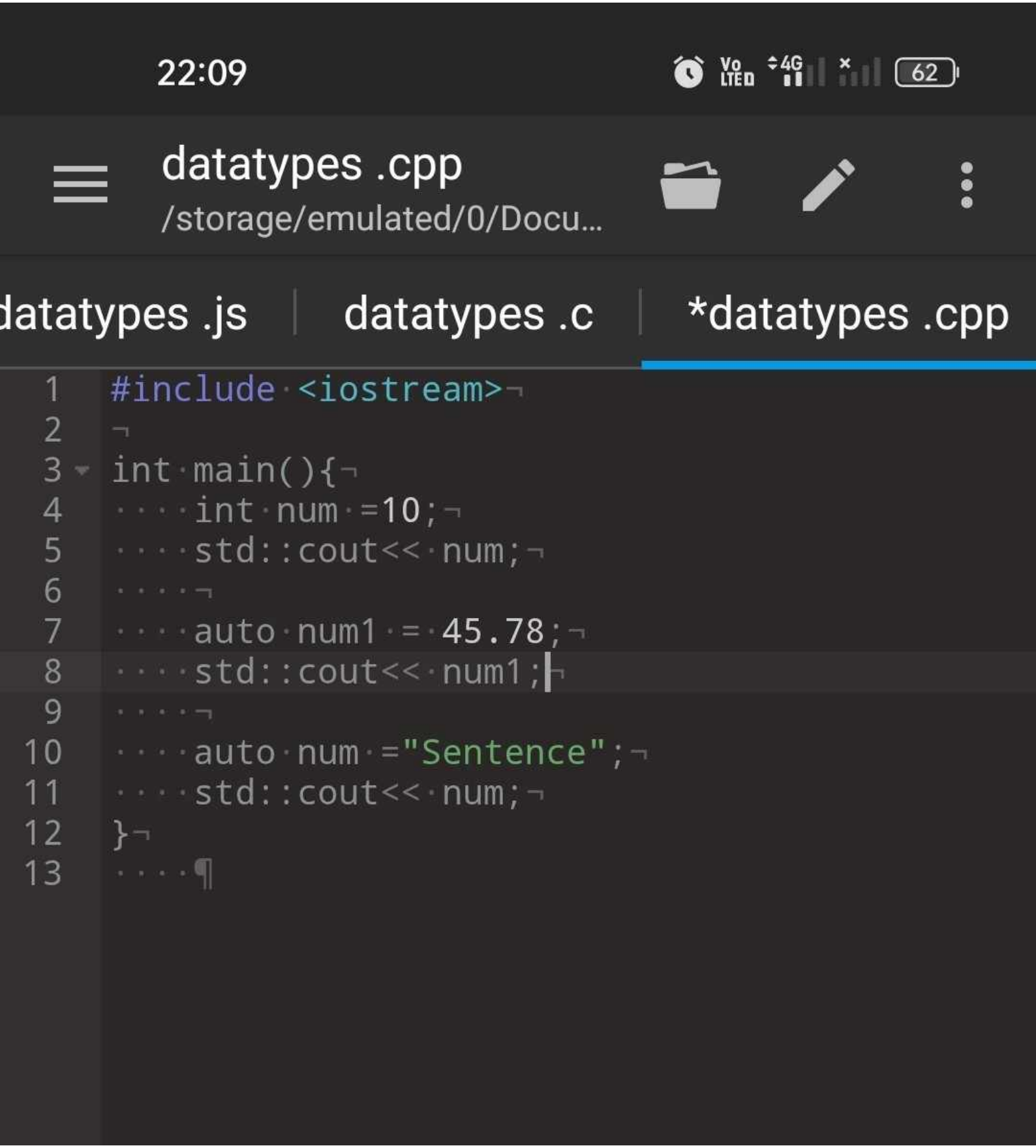

21:59
datatypes .c
/storage/emulated/0/Docu...
pattern .cpp
*datatypes .js
*datatypes .c
#include<stdio.h>
int main(){
int num =10;
printf("%d", num);
Error
=== compile output ===
main.cpp:7:11: error:
redefinition of 'num' with a
different type: 'float' vs
'int'
float num = 15.25;
^
main.cpp:4:9: note: previous
definition is here
int num =10;
^
1 error generated.
OK

# Error

```
=== compile output ===

main.cpp:10:10: error:
redefinition of 'num'
    auto num ="Sentence";
         ^
main.cpp:4:9: note: previous
definition is here
    int num =10;
        ^
1 error generated.
```

OK


22:09
datatypes .cpp
/storage/emulated/0/Docu...
datatypes .js
datatypes .c
*datatypes .cpp
#include <iostream>
int main(){
int num =10;
std::cout<< num;
auto num1 = 45.78;
std::cout<< num1;
auto num ="Sentence";
std::cout<< num;
}

```
ERROR!
Main.java:9: error: variable num is
    already defined in method main
    (String[])
ERROR!
        float num = 568.96;
              ^
Main.java:9: error: incompatible types:
    possible lossy conversion from double
    to float
        float num = 568.96;
                    ^
2 errors

=== Code Exited With Errors ===
```

```
1  // Online Java Compiler
2  // Use this editor to write, compile
       and run your Java code online
3
4  class Main {
5      public static void main(String[]
           args) {
6          int num =100;
7          System.out.println(num);
8
9          float num = 568.96;
10         System.out.println(num);
11         System.out.println("Try
               programiz.pro");
12     }
13 }
```